Code-Layout, Readability and Reusability

Maximum Marks: 3 Marks

Date	Team ID	Project Name
15 March 2026	Batch 2023-27 | ACOE	Rising Waters – ML-Based Flood Prediction System

Team Members

S.No	Team Member Name	Role
1	Venkatesh Balireddy	Team Lead
2	Archana Dhanani	Member
3	Shivatmika Gandikota	Member
4	Gode Siva Ramakrishna Durgaprasad	Member
5	Bavisetty Gopi Krishna	Member

Project Directory Structure

The Rising Waters project follows a modular, layered directory structure with clear separation of concerns between the ML pipeline, application layer, UI layer, and utilities.

File / Directory	Purpose	Author
app.py	Main Flask controller — all routes, session handling, prediction logging	Venkatesh Balireddy
model/train_model.py	Complete ML pipeline: preprocessing, training, evaluation, model save	Gode S R Durgaprasad
model/generate_dataset.py	Synthetic 5,000-row meteorological flood dataset generator	Shivatmika Gandikota
model/floods.save	Serialized XGBoost model (Joblib) — 96.55% test accuracy	All Members
model/scaler.save	Fitted StandardScaler object (Joblib) for input normalization	All Members
utils/preprocessing.py	Feature name constants, input validation, scaling helper functions	Archana Dhanani
templates/base.html	Shared Jinja2 layout: navigation, footer, CSS imports — zero duplication	Bavisetty Gopi Krishna
templates/result_flood.html	Dedicated Flood Chance result page — red theme, 6 emergency action cards	Bavisetty Gopi Krishna

File / Directory	Purpose	Author
templates/result_noflood.html	Dedicated No Flood result page — green theme, monitoring recommendations	Bavisetty Gopi Krishna
static/css/style.css	Premium dark-theme CSS with glassmorphism, animations, responsive layout	All Members
requirements.txt	All Python dependencies pinned to stable versions	Venkatesh Balireddy
Dockerfile	Docker container configuration for IBM Cloud deployment	Venkatesh Balireddy

Code Quality Standards Applied

S.No	Standard	Implementation in Rising Waters
1	Module Docstrings	Every Python file begins with a purpose docstring
2	Single Responsibility	Each function performs one task, max 40 lines
3	Constants	FEATURE_NAMES, MODEL_PATH defined at module level in utils/
4	Template Inheritance	All HTML pages extend base.html — zero repetition
5	CSS Variables	--primary-color, --accent-color CSS custom properties for theming
6	HTTP Status Codes	All Flask routes return appropriate 200/302/400 codes
7	Input Validation	Server-side validation in utils/preprocessing.py before model inference